

Capítulo 2 — GUÍA DE LECTURA

Capítulo 2 — GUÍA DE LECTURA.....	1
CSS: el modelo de caja, el flujo y la cascada	2
Las reglas de estilo, la geometría de la página y el algoritmo que desempata, explicados en criollo	2
Antes de empezar: cómo usar esta guía.....	2
2.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	3
La característica que separa a CSS de todo lo demás	3
2.2 — Por qué el estilo vive aparte del documento	4
Qué dice	4
En criollo.....	4
Las tres fallas, y lo que costaba cada una	4
Las tres propuestas, y por qué ganó la más pobre	4
Las cuatro decisiones que este capítulo desarrolla	5
La segunda decisión merece su propio párrafo	5
2.3 — Cómo el navegador decide el aspecto de un nodo	6
Qué dice	6
En criollo.....	6
2.4 — Anatomía de una regla	6
Acá se ve la cuarta decisión de diseño en acción	7
2.5 — La cascada: el algoritmo que desempata	8
2.5.1 — Origen e importancia	8
2.5.2 — Especificidad.....	9
Tres precisiones que se preguntan siempre	9
2.5.3 — Orden de aparición	10
2.5.4 — Herencia y valores iniciales	10
2.6 — El modelo de caja	10
2.6.1 — Las cuatro cajas	10
2.6.2 — <code>box-sizing</code> y el modelo «equivocado»	11
La ironía histórica, que es de las mejores del lenguaje	12
2.6.3 — Colapso de márgenes.....	13
2.7 — El flujo normal	13
2.7.1 — Bloque y línea	13
2.7.2 — Contextos de formato.....	14
2.7.3 — Posicionamiento	14
2.8 — Flexbox: repartir en un eje.....	15

2.9 — Grid: repartir en dos ejes	16
2.10 — Unidades y diseño adaptable	17
2.11 — Tailwind: qué problema resuelve el enfoque <i>utility-first</i>	18
2.11.1 — El problema de las hojas que sólo crecen	18
2.11.2 — Cómo funciona realmente	19
2.11.3 — Qué se gana y qué se pierde	20
2.12 — Herramientas de diagnóstico	20
2.13 — Seguridad y evolución	21
CSS puede exfiltrar información	21
CSS puede destruir la accesibilidad	22
Cuatro incorporaciones recientes, y qué problema cierra cada una	22
2.14 — Verificación: el checklist honesto	22
2.15 — Los once errores frecuentes	23
2.16 — Las actividades, y qué busca cada una	24
1. Auditoría de cascada	24
2. Especificidad a mano	24
3. El modelo de caja medido	24
4. Catálogo adaptable	24
5. Traducción de Tailwind	24
6. Exploración: la preferencia del usuario	24
7. Exploración: el costo del reordenamiento visual	25
2.17 — Síntesis: las once frases	25
2.18 — Qué leer, y en qué orden	25
Si lees una sola cosa	25
Si lees tres	26
Las fuentes normativas (para consultar, no para leer de corrido)	26
Cierre: las siete cosas que hay que recordar	26

CSS: el modelo de caja, el flujo y la cascada

Las reglas de estilo, la geometría de la página y el algoritmo que desempata, explicados en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce. La regla es la misma del Capítulo 1: **no se pierde ni un concepto**. Si el original dice *especificidad lexicográfica*, acá dice especificidad lexicográfica; si nombra RN-F06, acá está.

Cada sección conceptual tiene tres partes: **Qué dice** —la idea del original—, **En criollo** —con la analogía que la hace pegar— y **Para el pizarrón**. En las operativas se va directo.

LA IDEA MADRE DE TODO EL CAPÍTULO

Si te quedás con una sola frase, que sea esta:

CSS nunca falla ruidosamente. Cuando algo no se ve como esperabas, no hay error en ningún lado: hay una regla que ganó y vos no sabés cuál.

En JavaScript un error tira una excepción; en TypeScript el compilador se planta. En CSS no pasa nada. De ahí sale lo que gobierna todo el capítulo: **en CSS, saber diagnosticar importa más que saber escribir.**

2.1 — De qué se trata esta clase

Qué dice

El capítulo anterior terminó con el navegador combinando un árbol de nodos con un conjunto de reglas de estilo para producir el árbol de render. Este estudia esas reglas: qué son, cómo se resuelven cuando entran en conflicto y cómo determinan la geometría de cada caja. Y conviene desactivar de entrada una idea que hace mucho daño: **CSS no es la capa decorativa del sistema.**

En criollo

La creencia instalada es que CSS es lo lindo, lo que se hace al final. **Es falso, y es caro.** CSS decide si un formulario se puede usar con teclado, si un texto se lee con baja visión, si una tabla de pedidos entra en un teléfono. Y tiene traducción directa al TPI: la **sección 2.5 del TPI** —las once reglas obligatorias del frontend— no habla de colores, y sin embargo:

La regla del TPI	Qué exige	Por qué es presentación
RN-F06	Mostrar un estado de carga mientras la sesión se rehidrata	Ese estado es una caja que ocupa lugar, aparece y desaparece sin que la página salte, y un lector de pantalla tiene que anunciarla
RN-F11	Que la interfaz siga siendo utilizable cuando el canal de eventos se cae	«Utilizable» es que el aviso se vea, no tape los controles y lo desactualizado se distinga de lo que está al día

Las dos son reglas de negocio del enunciado y se resuelven acá. Si CSS fuera decoración, serían opcionales. No lo son.

La característica que separa a CSS de todo lo demás

CSS nunca falla ruidosamente. Una propiedad mal escrita no produce error: se descarta en silencio. Un selector que no coincide con nada no avisa. Un valor inválido se ignora y la declaración anterior queda en pie. Cuatro causas distintas y **un solo síntoma**: la pantalla se ve distinta, sin explicación. No es un descuido: es **la misma decisión de diseño de la sección 1.2** —la tolerancia al error de formato— aplicada a los estilos. **CSS es un buzón sin acuse de recibo.**

Al terminar la clase tenés que poder tomar una interfaz que no se ve como esperabas, abrir el inspector, **identificar qué regla ganó y por qué**, y corregirla sin `!important`.

PARA EL PIZARRÓN

El resto del módulo se aprende escribiendo. **CSS se aprende mirando.** No hay consola que te ayude, no hay compilador que te frene, no hay excepción que te apunte al renglón: lo único que tenés es el panel de estilos.

Por eso la pregunta no es «¿cómo hago para que se vea así?» sino «¿por qué se está viendo así?». La primera se contesta copiando de internet; la segunda, entendiendo el capítulo.

2.2 — Por qué el estilo vive aparte del documento

Qué dice

El HTML original no tenía forma de expresar apariencia, y no por olvido: la propuesta del CERN describía un formato para documentos estructurados, y cómo se veían era problema del programa que los mostrara. A mediados de los noventa esa premisa se rompió, y del desastre resultante salieron las decisiones de CSS.

En criollo

Al principio, que un documento se viera distinto en cada navegador **se consideraba correcto**. Duró poco: cuando la web dejó de usarse sólo para artículos académicos, quienes la usaban quisieron control sobre la apariencia, y la respuesta fue la peor posible: **meter la presentación adentro del HTML**. `<font>`, `<center>`, atributos como `bgcolor` y `border` por todas partes, y Netscape e Internet Explorer compitiendo con los suyos, incompatibles entre sí.

Las tres fallas, y lo que costaba cada una

La falla	Cómo se veía	Qué costaba
El documento dejó de ser estructura	Un título ya no era un <code><h2></code> : era un <code></code> en una celda de tabla	Para el navegador eso es texto en negrita, no un título : un lector de pantalla no podía anunciar una estructura que ya no existía
La maquetación con tablas	Tablas anidadas tres y cuatro niveles, rellenas de imágenes transparentes de un píxel estiradas para forzar separaciones: el <i>spacer gif</i>	Cambiar el ancho de una columna podía significar reescribir el documento entero
La duplicación sin límite	Un sitio de doscientas páginas repetía su tipografía miles de veces	Cambiar el color corporativo era un trabajo de días , y quedaban páginas sin actualizar

Las tres propuestas, y por qué ganó la más pobre

Håkon Wium Lie, entonces en el CERN, publicó en **octubre de 1994** la propuesta de *Cascading HTML Style Sheets*. La idea no era nueva —existía desde la composición tipográfica de los setenta— y había competencia. Y se repite el patrón del Capítulo 1: **ganó la más pobre**.

Propuesta	Qué ofrecía	Por qué no alcanzaba
DSSSL · para SGML	Un lenguaje de programación completo basado en Scheme	Había que saber programar en Scheme, y en 1996 la mayoría de los autores web no sabía programar en nada

Propuesta	Qué ofrecía	Por qué no alcanzaba
JSSS · de Netscape	Definir los estilos con JavaScript	Al ser código, no podía degradar con elegancia : lo que no se entendía se rompía
CSS · Lie, 1994	Un lenguaje declarativo : pares propiedad-valor y nada más	Es el menos potente de los tres, y ganó por eso

Las tres razones de la victoria explican su forma: era **declarativo** y se aprendía sin saber programar; **degradaba con elegancia**, así que se podían usar novedades sin romper los navegadores viejos; y **no requería que nadie migrara**.

CSS1 fue recomendación del W3C en diciembre de 1996; **CSS2** en 1998; **CSS2.1**, que corrigió lo anterior, recién en 2011. Desde entonces se desarrolla en **módulos independientes**, cada uno con su nivel, y por eso **no existe «CSS3» como especificación**: hay decenas de módulos, algunos en nivel 3, otros en 4 o 5.

Las cuatro decisiones que este capítulo desarrolla

Decisión	Qué gana	Qué cuesta
1. La cascada. Varias fuentes coexisten y sus conflictos se resuelven por un algoritmo declarado, no por el orden en que llegaron	Que el sistema sea predecible: dadas las mismas reglas, siempre gana la misma. La sección 2.5 lo estudia entero	Que haya que aprenderse el algoritmo : casi todos los problemas de CSS son gente peleándose contra un desempate que no conoce
2. El control repartido entre navegador, usuario y autor	Que una persona con baja visión pueda imponer sus preferencias sobre las del diseñador	Que vos no tengas la última palabra , y que sea facilísimo romper ese mecanismo sin querer
3. La herencia. Las propiedades tipográficas pasan de padre a hijo	Declararlas una vez en la raíz alcanza para todo el documento: es la respuesta a la tercera falla	Que a veces un elemento tenga un valor que nadie escribió ahí
4. La tolerancia al error. Lo inválido se descarta en silencio, declaración por declaración	Permitió que el lenguaje creciera treinta años sin romper nada	Un error de tipeo es invisible. Es la idea madre del capítulo

La segunda decisión merece su propio párrafo

CSS **no fue diseñado para que el autor tuviera el control total**, sino para repartirlo: el **navegador** aporta sus estilos por defecto, el **usuario** puede imponer los suyos, y el **autor** sos vos. La prueba de que el reparto va en serio está en un detalle que casi nadie conoce: **cuando el usuario marca una declaración como !important, esa declaración le gana a la del autor**, incluso si el autor también puso !important (tabla de la sección 2.5.1). Significa que una persona con baja visión que configura un tamaño mínimo de letra **tiene derecho** a que su preferencia gane. La consecuencia aparece en la sección 2.10: **fijar tamaños de fuente en píxeles rompe ese mecanismo**.

PARA ENTENDER: qué resignó CSS, y a favor de quién

La web resignó garantías **a cambio de escalar**. CSS resignó el control del autor **a cambio de que el usuario final pueda adaptar la página a sus necesidades**. Son dos formas de la misma pregunta, la que te tenés que hacer siempre: *¿qué resignó esto para funcionar, y a favor de quién?*

Acá la respuesta es fuerte: **el que tiene la última palabra sobre cómo se ve una página no sos vos, es quien la está leyendo**. Cada vez que escribas CSS que impide esa adaptación estás peleándote con una decisión de 1996, y vas a perder excluyendo gente.

2.3 — Cómo el navegador decide el aspecto de un nodo

Qué dice

La sección 1.7 describió el recorrido: el navegador construye el DOM a partir del HTML y el CSSOM a partir de las hojas de estilo, y de la combinación sale el árbol de render. Ese «combinar» tiene nombre propio: **el cálculo del valor de cada propiedad para cada elemento**.

En criollo

Para **cada nodo** y **cada una de las cientos de propiedades** que CSS define, el navegador llega a un **único valor final**, en cuatro etapas:

1. **Recolección**. Se juntan todas las declaraciones que aplican a ese elemento para esa propiedad, vengan de donde vengan.
2. **Cascada**. Si hay más de una, el conflicto se resuelve con el algoritmo de la sección 2.5, y queda **una sola ganadora**.
3. **Herencia o valor inicial**. Si no hubo ninguna, la propiedad toma el valor del padre —si es heredable— o su valor inicial.
4. **Cálculo**. El valor se convierte a algo absoluto: `2em` pasa a píxeles, `50%` se resuelve contra el contenedor.

Las cuatro de una: **es un concurso para cubrir un cargo**. Se juntan los postulantes, se elige uno con un reglamento escrito de antemano, si no se presentó nadie entra el hijo del anterior o el suplente de fábrica, y recién ahí se le pone el sueldo en pesos y no en «dos sueldos mínimos». De ahí, algo que conviene grabar: **todo elemento tiene un valor para toda propiedad, siempre**.

De dónde sale que un `<h1>` venga grande y en negrita

Un `<h1>` **no es grande por ser un `<h1>`**. No hay magia: hay una hoja que trae el navegador —la *user-agent stylesheet*— que declara ese aspecto. Es CSS, **y se ve en el inspector como cualquier otra regla**, abajo de todo, porque pierde contra todas las demás. Buscala una vez: la idea de que «los elementos vienen con aspecto» se te cae sola.

2.4 — Anatomía de una regla

Una hoja de estilos es una secuencia de reglas, y cada regla tiene esta forma:

```
.tarjeta-producto:hover {
  border-color: #2E74B5;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
}
```

Conviene aprenderse los nombres normativos de las partes: **son los que usa el inspector**.

Parte	En el ejemplo	Qué hace
Selector	<code>.tarjeta-producto:hover</code>	Determina a qué elementos aplica la regla
Bloque de declaraciones	<code>{ ... }</code>	Delimita el conjunto de declaraciones
Declaración	<code>border-color: #2E74B5;</code>	Una asignación de valor a una propiedad

Parte	En el ejemplo	Qué hace
Propiedad	<code>border-color</code>	Qué aspecto se está definiendo
Valor	<code>#2E74B5</code>	Qué valor toma

Se compone de **selectores simples** encadenados —acá, una clase y una pseudoclase—. Los que más se usan:

Tipo	Sintaxis	Coincide con
Universal	<code>*</code>	Cualquier elemento
De tipo	<code>button</code>	Todos los elementos de esa etiqueta
De clase	<code>.destacado</code>	Los que llevan esa clase
De identificador	<code>#carrito</code>	El que lleva ese id
De atributo	<code>[type="email"]</code>	Los que tienen ese atributo con ese valor
Pseudoclase	<code>:focus-visible</code>	Los que están en ese estado
Pseudoelemento	<code>::before</code>	Una parte generada del elemento

Y se relacionan mediante **combinadores**, que expresan la posición relativa en el árbol del DOM:

Combinador	Sintaxis	Significa
Descendiente	<code>nav a</code>	Un <code>a</code> en cualquier nivel dentro de un <code>nav</code>
Hijo	<code>ul > li</code>	Un <code>li</code> que es hijo directo de un <code>ul</code>
Hermano adyacente	<code>label + input</code>	El <code>input</code> inmediatamente después de un <code>label</code>
Hermano general	<code>h2 ~ p</code>	Cualquier <code>p</code> hermano posterior a un <code>h2</code>

Acá se ve la cuarta decisión de diseño en acción

Una declaración inválida se descarta sola, sin afectar a las demás:

```
.precio {
  color: #333333;
  font-size: 18píxeles; /* inválido: se descarta esta línea sola */
  font-weight: 600;     /* esta se aplica igual */
}
```

`18píxeles` no existe, y sin embargo `color` se aplica, `font-weight` se aplica y la regla sigue viva. **No hay error, no hay advertencia, no hay nada.** Dos caras: la buena, que podés escribir una propiedad nuevísima sin romper navegadores viejos, y es lo que permitió treinta años de evolución sin migraciones; la mala, que **un error de tipeo es completamente invisible**.

OJO ACÁ: los tres síntomas y sus tres causas

En CSS no hay mensajes de error. Nunca. No busques en la consola: no hay nada ahí. **Abrió el inspector y buscá la propiedad en el panel de estilos.** Vas a ver una de tres cosas:

- **No aparece.** Tu selector no coincidió, o escribiste mal el nombre.
- **Aparece tachada.** Otra regla le ganó en la cascada: andá a la sección 2.5.
- **Aparece con otro valor.** El valor que escribiste era inválido y se descartó.

Tres síntomas, tres causas, tres arreglos distintos. **Eso es diagnóstico, y es la mitad del trabajo con CSS.**

2.5 — La cascada: el algoritmo que desempata

Cuando varias declaraciones compiten por la misma propiedad del mismo elemento, el navegador aplica un algoritmo de desempate. Se evalúa en orden, y —esto es lo que más se olvida— **en cuanto un criterio decide, los siguientes no se consultan.**

Es el **desempate de una tabla de posiciones**: primero puntos, después diferencia de gol, después goles a favor. Por eso la especificidad —que todos creen que es lo primero— muchas veces ni se llega a mirar.

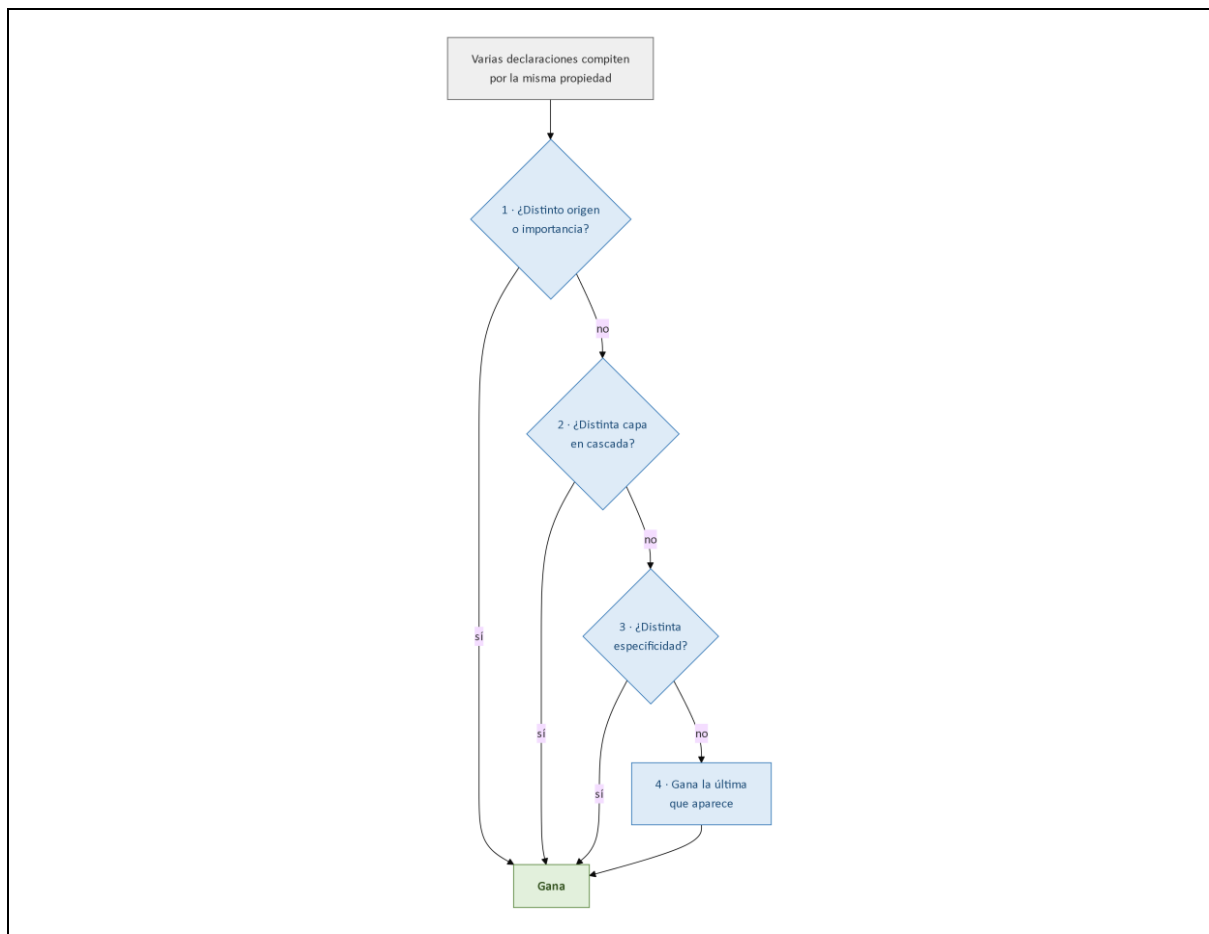


Figura 2.3. El orden de la cascada

2.5.1 — Origen e importancia

El primer criterio no es la especificidad, aunque casi todo el mundo lo crea: es **de dónde viene la declaración y si está marcada como importante**. Tres orígenes —navegador, usuario y autor— por dos niveles de importancia dan seis escalones, de menor a mayor prioridad:

Prioridad	Origen	Importancia
1 (más débil)	Navegador	normal
2	Usuario	normal
3	Autor	normal
4	Autor	!important
5	Usuario	!important

Prioridad	Origen	Importancia
6 (más fuerte)	Navegador	!important

Mirá lo que pasa entre los niveles 3 y 5: **con !important el orden se invierte**. En condiciones normales el autor le gana al usuario; cuando los dos marcan como importante, **gana el usuario**. Es la segunda decisión de diseño de la sección 2.2 **escrita adentro del algoritmo**. Y guardate la tabla, porque **es el criterio que más veces resuelve el conflicto sin que la especificidad llegue a jugar**.

2.5.2 — Especificidad

Si el criterio anterior no alcanzó, se compara la **especificidad** del selector: un vector de tres números, de izquierda a derecha, que sale de contar tres cosas:

Componente	Qué cuenta
A	Selectores de identificador (#carrito)
B	Clases, atributos y pseudoclases (.activo, [type], :hover)
C	Tipos y pseudoelementos (div, ::before)

Y acá viene lo que rompe la intuición: la comparación es **lexicográfica, no numérica**. Se mira A; si difiere, ya está decidido. **Un solo identificador le gana a cualquier cantidad de clases**.

Selector	A	B	C	Comentario
p	0	0	1	
.destacado	0	1	0	Le gana a cualquier cantidad de tipos
nav ul li a	0	0	4	Cuatro tipos pierden con una clase
.menu .item.activo	0	3	0	
#carrito	1	0	0	Le gana a las tres clases anteriores
style="..."	—	—	—	Los estilos en línea ganan a todo

Tres precisiones que se preguntan siempre

- El universal * y los combinadores **no aportan especificidad**: `ul > li` vale lo mismo que `ul li`.
- `:not()` no aporta, pero **sí su argumento**: `:not(.activo)` vale una clase.
- `:is()` toma la especificidad de su argumento más específico; **`:where()` siempre vale cero**, y esa es su razón de existir: reglas fáciles de sobrescribir.

OJO ACÁ: la especificidad no se suma

Se compara de a columnas, y la primera que difiere decide. Mirá:

- `body div.contenedor ul li a.enlace.activo` → (0, 3, 4)
- `#menu` → (1, 0, 0)

Siete selectores contra uno, y **gana el segundo** antes de que la comparación llegue a mirar las clases. Es como comparar 0,34 con 1,00: por más decimales que agregues, nunca pasás el 1. Por eso pelearle a un #identificador con clases es perder el tiempo: **te dejan sin margen de maniobra hacia arriba**.

2.5.3 — Orden de aparición

Si dos declaraciones empatan en origen, importancia y especificidad, **gana la última que aparece**. De ahí, dos cosas. **Una:** el orden de los archivos importa, y por eso «poné tu hoja después de la de la biblioteca» no es superstición. **Dos:** la regla práctica es **cargar lo general antes que lo particular** — base, componente, página—, porque al revés vas a necesitar especificidad, o peor `!important`, para algo que se arreglaba moviendo un `<link>` de lugar.

2.5.4 — Herencia y valores iniciales

Cuando ninguna declaración aplica a un elemento, la propiedad toma su valor por la herencia o por el valor inicial definido por la especificación. El reparto no es arbitrario:

Camino	Cuáles	Por qué
Se hereda del padre	Las tipográficas: <code>color</code> , <code>font-family</code> , <code>font-size</code> , <code>line-height</code> , <code>text-align</code> , <code>visibility</code>	Declararlas una vez en <code>html</code> o <code>body</code> alcanza para todo el documento: la respuesta a la tercera falla de la sección 2.2
Toma su valor inicial	Las de caja y disposición: <code>margin</code> , <code>padding</code> , <code>border</code> , <code>width</code> , <code>display</code> , <code>background</code>	Si el <code>padding</code> se heredara, un contenedor con relleno se lo impondría a todos sus descendientes, nivel por nivel

Por eso la caja no se hereda y la letra sí.

OJO ACÁ: `!important` es deuda, no solución

Casi nunca lo necesitás, y usarlo es endeudarte. La secuencia es siempre la misma: ponés uno para ganarle a una regla que no entendés, semanas después alguien necesita sobrescribir la tuya y pone otro, y así hasta que la hoja es una pila de importantes donde no se puede cambiar nada sin romper otra cosa.

Cuando sientas la necesidad, **abrí el inspector y averiguá qué regla te está ganando y por qué:** el noventa por ciento de las veces la solución es arreglar un selector. La excepción legítima existe — sobrescribir una biblioteca de terceros que no podés editar—, y ahí es una herramienta, no una deuda.

2.6 — El modelo de caja

2.6.1 — Las cuatro cajas

Todo elemento del árbol de render genera al menos una caja rectangular con cuatro áreas concéntricas, que se comportan distinto frente al fondo:

Área	Delimitada por	Contiene	Se ve el fondo
Contenido	<code>width</code> , <code>height</code>	El texto o los hijos	Sí
Relleno (<i>padding</i>)	<code>padding</code>	Espacio interior	Sí
Borde	<code>border</code>	La línea del borde	El borde mismo
Margen	<code>margin</code>	Espacio exterior	No, siempre transparente

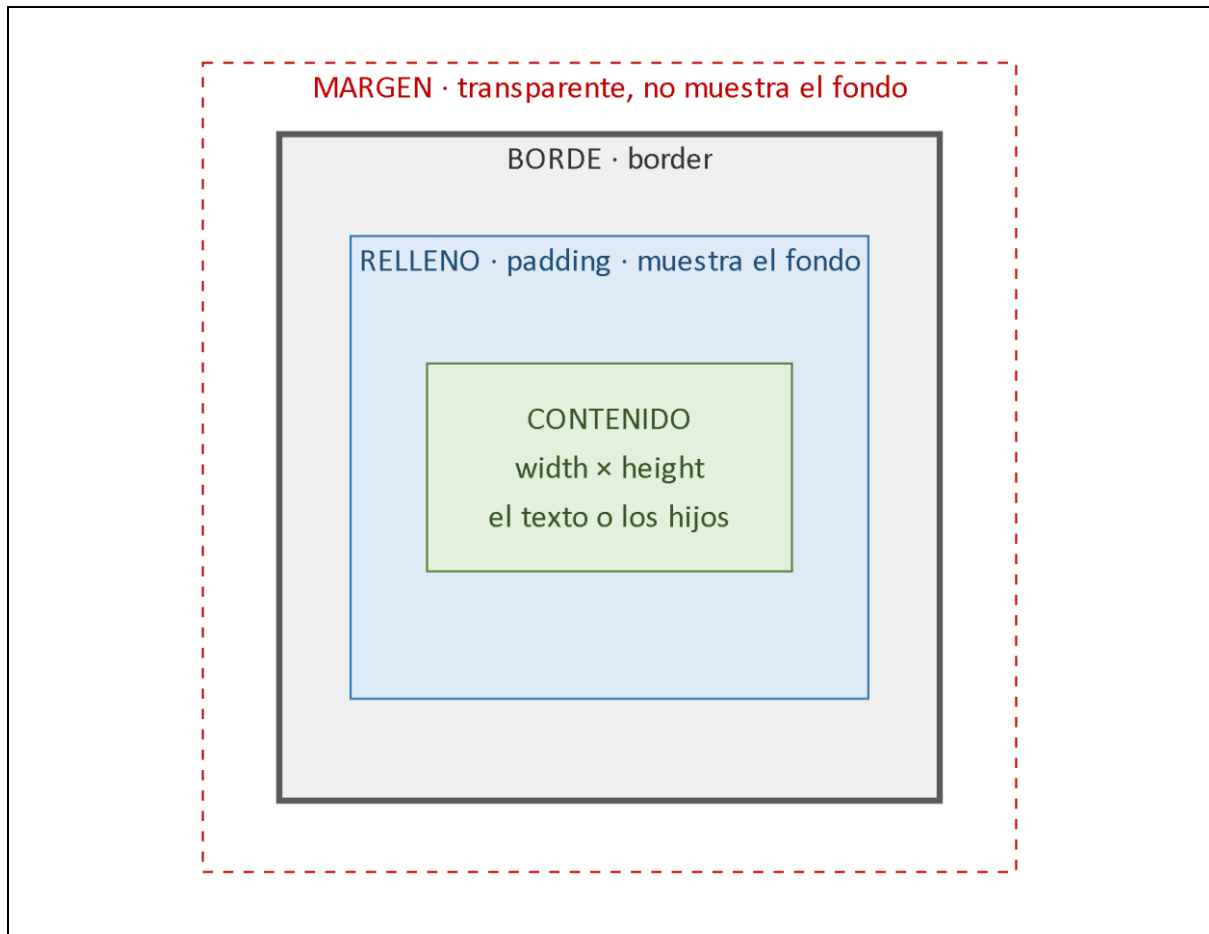


Figura 2.1. Las cuatro áreas del modelo de caja

La distinción entre relleno y margen es la que más se confunde, y el criterio es preciso: **el relleno es espacio de adentro, el margen es espacio de afuera**. De ahí, dos cosas para diagnosticar: un espacio con el color del elemento es relleno y uno transparente es margen; y **sólo los márgenes colapsan**.

PARA ENTENDER: la campera y la distancia

El relleno es lo abrigado que estás. El margen es cuánto te alejás del otro.

El relleno es tu campera: es tuya, y si te pintan de rojo también sale roja — por eso el fondo se ve ahí. El margen es el espacio que dejás con la persona de al lado: no tiene color, **es de los dos**, y por eso cuando los dos piden espacio no se suma, se acuerda uno solo. Eso es el colapso de márgenes.

2.6.2 — box-sizing y el modelo «equivocado»

Por defecto, `width` define el ancho del **área de contenido solamente**: el relleno y el borde se suman por fuera. Es correcto según CSS1 y es la trampa más conocida del lenguaje:

```
.tarjeta {
  width: 300px;
  padding: 20px;
  border: 2px solid #999999;
}
```

Eso ocupa $300 + 20 + 20 + 2 + 2 = 344$ píxeles. Pediste 300 y ocupa 344: **no es un bug, es el estándar**. `width` describe el hueco de adentro, no lo que la caja mide.

Es una caja de mudanza. «Esta caja es de 30 centímetros» ¿habla del hueco donde entran tus cosas o de lo que ocupa en el camión? `content-box` dice el hueco; `border-box`, lo que ocupa en el camión. Y el que dice «esta caja mide 300 píxeles» piensa en lo que ocupa: por eso el modelo por defecto **contesta la pregunta que nadie hizo**.

La ironía histórica, que es de las mejores del lenguaje

Internet Explorer 5, en su modo de compatibilidad, implementó el modelo **al revés**: `width` incluía relleno y borde. Con el tiempo quedó claro que ese «error» era el modelo que la gente quería, y CSS3 lo incorporó como opción:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Con eso `width` pasa a incluir relleno y borde, y la tarjeta mide exactamente 300 píxeles. Los pseudoelementos van incluidos a propósito: si no, `::before` y `::after` quedan con el modelo viejo y desbordan.

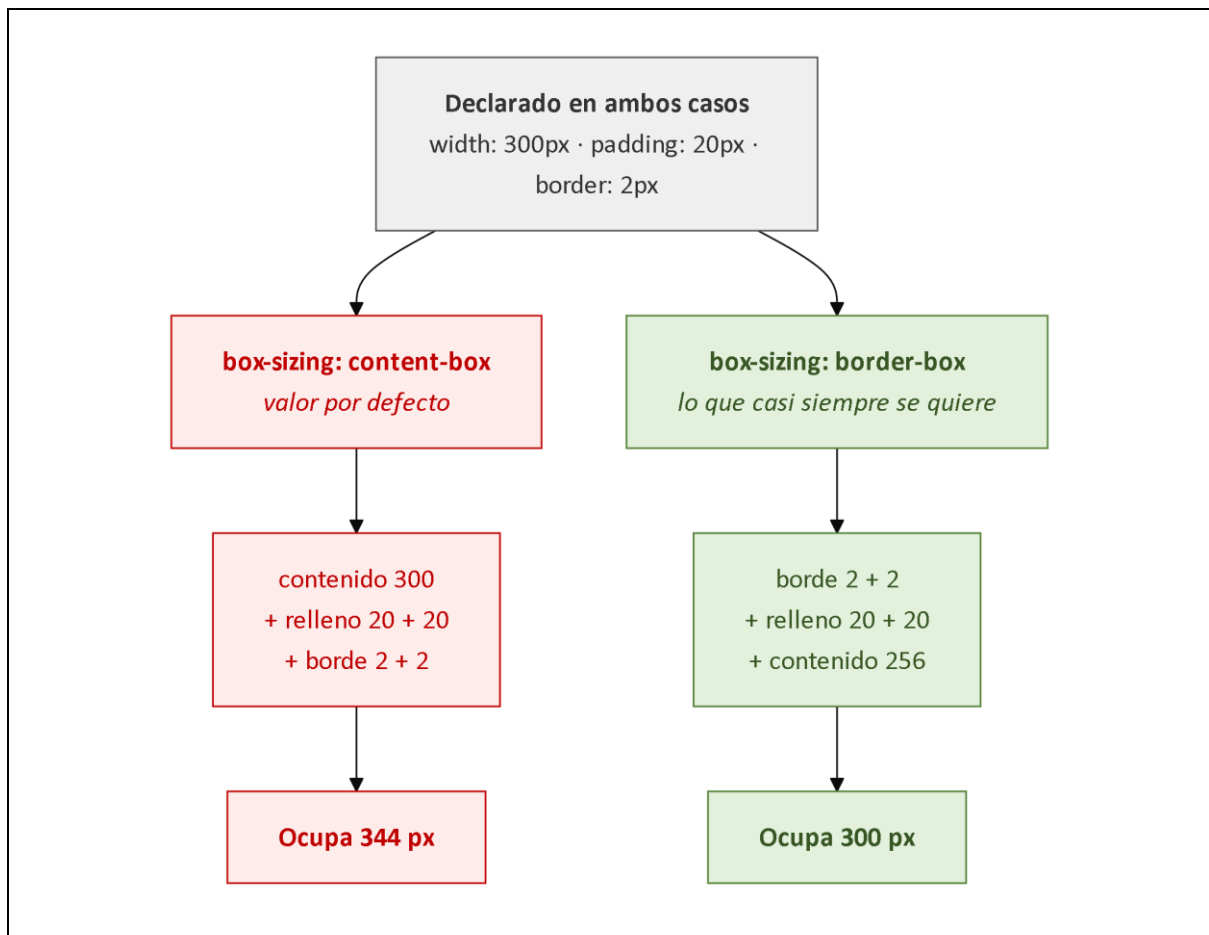


Figura 2.2. `content-box` frente a `border-box`

Todo proyecto moderno incluye esa regla, y **Tailwind la trae en su capa base**: en el TPI la vas a tener puesta sin haberla escrito. Vale la pena saber que está ahí, porque **es la primera línea de todo lo que se va a maquetar**.

Cuando el estándar se equivoca y el bug tiene razón

El modelo que hoy usa todo el mundo era el «error» de Internet Explorer. Durante años border-box fue *el bug de Microsoft*, y hoy no vas a encontrar un proyecto serio que no lo active en la primera línea.

¿Y qué te llevás? Que **«cumple el estándar» y «es lo correcto» no siempre coinciden.** Saber cuál es criterio, y eso no te lo da la documentación.

2.6.3 — Colapso de márgenes

Los márgenes verticales adyacentes **no se suman: se fusionan**, y el resultado es el mayor de los dos: un párrafo con 20 píxeles de margen inferior seguido de otro con 30 de margen superior quedan separados por **30**, no por 50. El comportamiento hereda de la composición tipográfica, donde el espacio entre dos bloques es una propiedad de la relación entre ellos y no la suma de dos pedidos: **es uno solo** y los dos lo están pidiendo. Ocurre en tres situaciones:

Dónde colapsa	Qué se ve
Entre hermanos adyacentes	La separación es el mayor de los dos márgenes, no la suma. El caso benigno: casi siempre es lo que querías
Entre un padre y su primer o último hijo	El margen del hijo «se escapa» y mueve al padre entero. El que desconcierta
En un elemento vacío , consigo mismo	Sus márgenes superior e inferior se fusionan en uno

Se interrumpe de tres maneras: **relleno** en el padre, **borde** en el padre, o **un contexto de formato nuevo** —display: flex, grid o flow-root—. Y ahí está la razón por la que en un contenedor flex los márgenes ya no colapsan: **el colapso es una propiedad del flujo normal, y Flexbox no es flujo normal**. No es que lo «desactive»: nunca existió fuera del flujo normal.

EXPERIMENTO — el margen que se escapa

El colapso entre padre e hijo desconcierta porque parece que el margen se escapara de la caja. Comprálo:

- Hacé un div con fondo de color y adentro un <p> con margin-top: 40px.
- El que se movió hacia abajo es el div entero**, no el párrafo: el margen del hijo salió por arriba del padre.
- Agregale al div un padding-top: 1px: el párrafo se acomoda adentro y el div deja de moverse.
- Sacá el relleno y probá con display: flow-root. Mismo resultado, sin un píxel de nada.

Si no hay nada entre el borde del padre y el margen del hijo —ni relleno, ni borde, ni un contexto nuevo—, para el motor de disposición **son el mismo margen**. Cuando veas un espacio que no pusiste, **sospechá de esto primero**.

2.7 — El flujo normal

2.7.1 — Bloque y línea

El **flujo normal** es la disposición por defecto, y tiene dos comportamientos. Los elementos de **bloque** —div, p, h1, section— ocupan todo el ancho, se apilan uno debajo del otro y aceptan dimensiones y márgenes en las cuatro direcciones: **son cajones apilados**.

Los elementos **en línea** —`span`, `a`, `strong`, `em`— ocupan sólo lo que su contenido necesita y siguen el sentido del texto: **son palabras dentro de un renglón**. Y tienen dos restricciones que sorprenden:

La restricción	Qué pasa si la ignorás
Ignoran <code>width</code> y <code>height</code>	Escribís <code>width: 200px</code> en un <code></code> y no pasa nada: la declaración aparece aplicada en el inspector y no tiene efecto
Aceptan márgenes horizontales, no verticales	Un <code></code> con <code>margin-top: 40px</code> no se mueve ; el izquierdo y el derecho sí funcionan

La razón se reconstruye sola: un elemento en línea es parte del renglón, y la altura del renglón la determina la tipografía; que un fragmento de texto la empuje rompería la composición del párrafo. Pedirle a un `<span>` que mida 200 píxeles de alto es como pedirle a la palabra «milanesa» que se levante del renglón. Cuando hacen falta dimensiones verticales existe `display: inline-block`: **por afuera palabra, por adentro cajón**.

2.7.2 — Contextos de formato

Un **contexto de formato** es una región del documento con sus propias reglas de disposición: el flujo normal es uno, y Flexbox y Grid crean otros. Explica de una vez tres comportamientos que sin él parecen arbitrarios: por qué los márgenes dejan de colapsar dentro de un contenedor flex (sección 2.6.3), por qué un flotante deja de desbordar cuando se establece un contexto nuevo, y por qué `display` cambia tantas cosas a la vez. La frase que hay que llevarse: **display no cambia una propiedad: cambia el conjunto de reglas bajo el cual el elemento y sus hijos se disponen**. Es **cambiar el deporte que se juega en la cancha**: si pasás de fútbol a básquet no cambia una regla, cambian todas.

2.7.3 — Posicionamiento

La propiedad `position` saca un elemento del flujo o lo desplaza respecto de él, y dos preguntas separan sus cinco valores: **¿sigue ocupando su lugar?** y **¿contra qué se mide el desplazamiento?**

Valor	¿Sigue en el flujo?	Referencia del desplazamiento
<code>static</code>	Sí	No admite desplazamiento; es el valor por defecto
<code>relative</code>	Sí	Su propia posición original
<code>absolute</code>	No	El ancestro posicionado más cercano
<code>fixed</code>	No	La ventana del navegador
<code>sticky</code>	Sí	Alterna entre relativo y fijo según el desplazamiento

La distinción entre `relative` y `absolute` es la que más consecuencias tiene. `relative` **desplaza el elemento pero le conserva su lugar en el flujo**: el hueco sigue ahí, como si se moviera y su sombra se quedara ocupando el asiento. `absolute` lo saca del flujo, y los demás se acomodan como si no existiera. De ahí el patrón más usado: un contenedor con `position: relative` **que no se mueve** y existe sólo para servir de referencia a un hijo `absolute`.

PARA ENTENDER: cada ``absolute`` necesita su ``relative``

Ese `position: relative` que no mueve nada parece código al pedo. **No lo es: es lo más importante de la regla**. Es el chinche y el corcho: un `absolute` busca hacia arriba, padre por

padre, hasta el primero con `position` distinto de `static`, y si no encuentra ninguno se clava en la pared del fondo — el documento.

Por eso el síntoma clásico es tan raro: el **badge del carrito aparece arriba a la izquierda de toda la página en vez de en la esquina del ícono**. No está roto el `top: 0`; `right: 0`: está midiendo contra la página.

Regla mental: **cada `absolute` necesita su `relative`**. Si el hijo se te fue de viaje, fijate qué le falta al padre.

2.8 — Flexbox: repartir en un eje

Flexbox resuelve la distribución de elementos **en un solo eje**. Nació para reemplazar los flotantes, que se usaban para maquetar aunque estaban diseñados para **hacer que el texto rodee una imagen**, como en una revista.

Se organiza alrededor de dos ejes: el **eje principal**, cuya dirección define `flex-direction`, y el **eje cruzado**, perpendicular. La alineación se refiere **a los ejes, no a «horizontal» y «vertical»**: `justify-content` alinea sobre el principal y `align-items` sobre el cruzado, así que con `column` las dos **intercambian su efecto visual**. Es **una sogá de ropa**: una reparte las prendas a lo largo de la sogá y la otra decide a qué altura cuelga cada una.



Figura 2.4. Los ejes de Flexbox

Propiedad	Qué controla	Valores
<code>flex-direction</code>	Dirección del eje principal	<code>row</code> , <code>column</code>
<code>justify-content</code>	Distribución en el eje principal	<code>flex-start</code> , <code>center</code> , <code>space-between</code>

Propiedad	Qué controla	Valores
<code>align-items</code>	Alineación en el eje cruzado	<code>stretch</code> , <code>center</code> , <code>flex-start</code>
<code>flex-wrap</code>	Si los elementos saltan de línea	<code>nowrap</code> , <code>wrap</code>
<code>gap</code>	Separación entre elementos	Cualquier longitud
<code>flex-grow (hijo)</code>	Cuánto crece si sobra espacio	inicial <code>0</code>
<code>flex-shrink (hijo)</code>	Cuánto se encoge si falta	inicial <code>1</code>
<code>flex-basis (hijo)</code>	Tamaño de partida antes de repartir	inicial <code>auto</code>

Las tres del hijo se resumen en la abreviatura `flex`, y sus valores iniciales explican el comportamiento por defecto: **no crece** si sobra espacio y **sí se achica** si falta.

`gap` merece una mención aparte: antes de existir, separar elementos exigía margen en todos y quitárselo al último con `:last-child` —feo porque **el margen es del hijo, y la separación es de la relación entre hijos**—. `gap` declara la separación **entre** elementos, sin agregar nada en los extremos.

OJO ACÁ: ``justify`` y ``align`` no son «horizontal» y «vertical»

`justify-content` es «a lo largo del eje principal»; `align-items`, «a lo ancho del cruzado». Que en row coincidan con horizontal y vertical es una **casualidad del caso más común**, no una definición: el día que pongas `column` y `align-items`: `center` te centre horizontalmente, **rotaste la sogá**. Antes de escribir la propiedad, preguntate **dónde está el eje principal ahora**.

2.9 — Grid: repartir en dos ejes

Grid resuelve la distribución **en dos ejes simultáneos**, y es el **reemplazo legítimo de la maquetación con tablas de la sección 2.2**: hace lo mismo, sin tocar la estructura del documento. La diferencia con Flexbox no es «uno para poco y el otro para mucho»:

	Flexbox	Grid
De dónde parte	Del contenido : los elementos se acomodan según su tamaño y el espacio disponible	Del contenedor : primero se define la grilla, después se colocan los elementos
Dimensiones	Una	Dos
La imagen	Acomodar sillas alrededor de una mesa según cuánta gente vino	El plano del aula con los pupitres ya marcados en el piso

Se usan juntos: **Grid para la disposición general de la página**, **Flexbox para el interior de cada componente**.

La pregunta que decide cuál usar

No te preguntes cuál es más moderno ni más potente, sino una sola cosa: **¿quién manda sobre el tamaño, el contenido o vos?**

- Si querés que se acomoden según lo que miden —una barra donde cada ítem ocupa lo que dice su texto— **eso es Flexbox**.
- Si querés que entren en una estructura definida de antemano —un catálogo donde todas las tarjetas miden igual— **eso es Grid**.

Y ojo con «Flexbox es para cosas chicas»: no es cuestión de escala sino de **quién decide la medida**. Podés tener un Grid de dos celdas y un Flexbox de cuarenta elementos, y estar bien en los dos casos.

```
.catalogo {
  display: grid;
```



```
grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
gap: 1rem;
}
```

Esas tres líneas producen una grilla adaptable **sin una sola consulta de medios**:

- **1fr** es una unidad propia de Grid que significa «una fracción del espacio sobrante», repartido después de descontar lo fijo.
- **minmax(220px, 1fr)** declara que cada columna mide como mínimo 220 píxeles y como máximo una fracción del sobrante.
- **repeat(auto-fill, ...)** crea tantas columnas como entren, calculado por el navegador según el ancho real del contenedor.

Pensá lo que pasó: **le delegaste la decisión al navegador**. En vez de «a 768 píxeles, dos columnas», declararás la restricción y él resuelve cada ancho posible, **incluidos los que vos no probaste**.

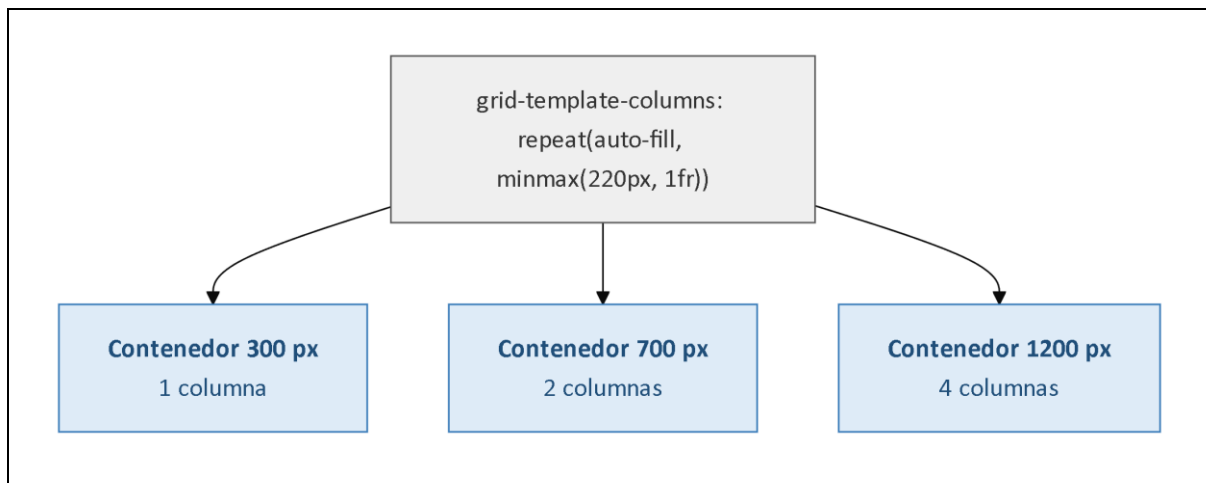


Figura 2.5. Grid adaptable con `auto-fill` y `minmax`

2.10 — Unidades y diseño adaptable

La elección de unidad **no es estilística**: determina si el diseño responde a las preferencias del usuario o las ignora. Es donde la segunda decisión de la sección 2.2 se vuelve una línea de código.

Unidad	Relativa a	Uso recomendado
px	Nada; es absoluta	Bordes, sombras, detalles que no deben escalar
rem	Tamaño de fuente de la raíz	Tipografía y espaciados
em	Tamaño de fuente del propio elemento	Espaciados que deben acompañar al texto
%	La misma propiedad del contenedor	Anchos dentro de un contenedor
vw / vh	Ancho / alto de la ventana	Secciones a pantalla completa
ch	Ancho del carácter «O»	Ancho de columnas de texto

Todo navegador permite configurar un tamaño de fuente base. Por defecto son **16 píxeles**, y una persona con dificultades de visión lo sube. **Un tamaño declarado en rem se multiplica por esa base y respeta la preferencia; uno declarado en px la ignora por completo**. Es un plano a escala contra un plano con las medidas quemadas en centímetros: si agrandás la hoja, el primero crece entero; el otro no crece, o peor, crece la mitad y el dibujo se rompe.

Poner `font-size: 14px` en el cuerpo de un documento es **decidir que la preferencia de accesibilidad de esa persona no se aplica en tu sitio**. Rara vez es consciente, y casi siempre es lo que termina pasando.

EXPERIMENTO — subí la letra del navegador

Esto lo vas a recordar toda la carrera, así que hacelo de verdad.

1. Buscá el tamaño de fuente en la configuración de tu navegador y subilo a 24 píxeles, que es lo que usa mucha gente con baja visión.
2. Navegá tres o cuatro sitios que uses todos los días.

Vas a ver tres comportamientos. Algunos sitios agrandan todo y se leen perfecto: usaron `rem`. Otros no cambian nada: usaron `px`. Y otros agrandan **sólo una parte** y se rompen, con textos fuera de sus cajas y botones pisados: mezclaron las dos unidades sin darse cuenta.

1. Anotá cuál es cuál. Volvé a 16 antes de seguir.

No es un detalle de accesibilidad para una minoría: **es lo que le pasa todos los días a alguien que necesita letra más grande**, y es la diferencia entre poder comprar en un sitio o no poder.

Las **consultas de medios** aplican reglas según características del dispositivo, y la práctica establecida es escribir primero para pantalla angosta y agregar reglas hacia arriba: **el diseño angosto es el caso restrictivo, y partir de él obliga a resolver lo difícil primero**.

```
.catalogo { grid-template-columns: 1fr; }

@media (min-width: 48rem) {
  .catalogo { grid-template-columns: repeat(2, 1fr); }
}
```

Existe además una consulta que no habla del tamaño sino de **una preferencia declarada del usuario**, que conviene incorporar como hábito:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

Para muchas personas con trastornos vestibulares, una animación de desplazamiento **produce mareo real** —náuseas, no incomodidad estética—, y esa consulta lee una preferencia que ya configuraron en su sistema operativo. Y **es de los pocos lugares donde `!important` está justificado**, porque tiene que ganarle a cualquier animación declarada después: la excepción legítima de la sección 2.5.4.

2.11 — Tailwind: qué problema resuelve el enfoque *utility-first*

El stack del TPI declara **Tailwind CSS 3.x**, y conviene entender qué vino a resolver, porque **el problema no es el que suele suponerse**: no vino a hacer las páginas más lindas ni a ahorrarte CSS, sino a resolver el crecimiento sin límite de las hojas de estilo.

2.11.1 — El problema de las hojas que sólo crecen

Durante años la buena práctica fue el **CSS semántico**: nombrar las clases según **lo que el elemento es**, no según cómo se ve. Metodologías como **BEM** formalizaron el enfoque, y la promesa era

reutilización y mantenibilidad. En proyectos grandes el resultado fue otro, en tres síntomas que se alimentan entre sí:

El síntoma	Por qué pasa
Nadie borra CSS. Se elimina un componente y su CSS queda: la hoja sólo crece	Borrarlo exige estar seguro de que nadie más lo usa, y esa certeza es difícil. Ante la duda, se deja
Nombrar cuesta. <code>.tarjeta</code> , <code>.item-catalogo</code> o <code>.producto-preview</code>	Nunca hay respuesta correcta, y cuando el diseño cambia el nombre queda mintiendo : una <code>.barra-lateral</code> que ahora es una fila superior
Modificar da miedo. En vez de tocar una regla, se agrega otra más específica	Cambiar una regla afecta lugares que no tenés a la vista, y la defensa refuerza el primer síntoma

La analogía del primero la tenés en tu casa: **es el cajón de los cables**. Nadie tira ninguno por las dudas, y llega un punto en que buscar ahí cuesta más que comprarlo de nuevo.

Adam Wathan publicó Tailwind en **2017** invirtiendo la premisa: **las clases no describen qué es el elemento, describen cómo se ve**.

```
<article class="rounded-lg border border-slate-200 p-4 shadow-sm">
  <h3 class="text-lg font-semibold text-slate-900">Milanesa napolitana</h3>
  <p class="mt-1 text-sm text-slate-600">Con papas fritas</p>
</article>
```

Las tres consecuencias son **el reverso exacto de los tres síntomas**. **La hoja deja de crecer**, porque el conjunto de utilidades es finito y se comparte: agregar un componente no agrega CSS. **No hay que nombrar nada**. Y **modificar deja de dar miedo**, porque el alcance de un cambio es el elemento que estás editando.

2.11.2 — Cómo funciona realmente

Tailwind no es una hoja de estilos que se enlaza: es una herramienta que corre durante la construcción, en tres pasos.

1. **Escanea** los archivos de origen configurados buscando texto que parezca un nombre de clase.
2. **Genera** únicamente el CSS de las clases que encontró.
3. **Emite** una hoja que en producción pesa unos pocos kilobytes.

Del paso 1 se desprende una limitación que conviene saber **de antemano**: **el escaneo es textual, no ejecuta el código**. Una clase construida por concatenación en tiempo de ejecución no se encuentra, y su CSS no se genera:

```
// NO funciona: la cadena completa no existe en el archivo
const clase = `text-${color}-500`;

// Sí funciona: las dos cadenas completas están escritas
const clase = activo ? "text-green-500" : "text-slate-500";
```

En el segundo caso las cadenas están escritas enteras y el escaneo las encuentra; en el primero, `text-green-500` **no existe en ningún lado** hasta que el programa corre, y para entonces la hoja ya se generó.

Es el error número uno de quien empieza con Tailwind, y desconcierta porque **funciona en desarrollo y falla en producción**. El diagnóstico no lleva un minuto: **buscá la clase en el CSS generado**. Si no está, el problema es el escaneo, no el estilo.

2.11.3 — Qué se gana y qué se pierde

Fiel a la pregunta del Capítulo 1 —¿qué resignó para funcionar?—, corresponde enunciar lo que Tailwind resigna:

Lo que se resigna	Qué significa en la práctica
El marcado se vuelve ruidoso	Un elemento con doce utilidades es más difícil de leer que uno con una clase. Es una crítica válida y no tiene refutación: se acepta a cambio de lo demás
La abstracción se muda al componente	Cuando una combinación se repite, la solución no es crear una clase sino extraer un componente . Funciona en un proyecto con componentes, y no funciona en uno que no los tiene
Aparece una dependencia de construcción	Sin la herramienta ejecutándose, no hay estilos. Es el mismo intercambio que el Capítulo 7 va a estudiar con Vite

Tailwind no te exige de saber CSS

Tailwind **no te exige de entender CSS**: te exige de nombrar cosas. `flex items-center justify-between` es exactamente `display: flex; align-items: center; justify-content: space-between`. Si no entendiste la sección 2.8, son tres palabras mágicas que copias y no sabés por qué a veces funcionan.

Y esto importa ahora que vas a trabajar con agentes de IA: un agente te va a escribir Tailwind sin dudar y le va a salir plausible. Para saber si está bien —si ese `items-center` alinea sobre el eje que querés, si ese `px` debería ser `rem`, si ese contraste alcanza— **tenés que saber CSS**. La herramienta te ahorra tipeo, no criterio.

2.12 — Herramientas de diagnóstico

El panel de elementos del navegador es **el instrumento central de este capítulo**: como CSS no emite errores, es el único lugar donde el lenguaje te cuenta qué hizo. Cuatro zonas concentran lo necesario.

La zona	Qué muestra	Qué contesta
El panel de estilos	Las reglas que aplican al elemento ordenadas por prioridad de la cascada : la ganadora arriba, las perdedoras tachadas , cada una con su archivo y línea, y las del navegador identificadas como tales	¿Qué regla ganó y por qué? Es el algoritmo de la sección 2.5, dibujado
Los valores calculados	El valor final de cada propiedad tras las cuatro etapas de la sección 2.3	¿Cuánto mide realmente este <code>2em</code> ?
El diagrama del modelo de caja	Las cuatro áreas de la sección 2.6 con sus medidas reales	¿Ese espacio es margen o relleno?

La zona	Qué muestra	Qué contesta
Las superposiciones de Flexbox y Grid	Las líneas de la grilla y los ejes del contenedor dibujados sobre la página; para Grid, además, numera las líneas	<i>¿Por qué este elemento cayó donde cayó?</i>

FIGURA 2.6

captura

Captura del panel con un elemento que recibe la misma propiedad de varias reglas: la ganadora arriba y las perdedoras tachadas. Debe verse también el archivo y la línea de origen de cada regla

Figura 2.6. El panel de estilos y la cascada

FIGURA 2.7

captura

Captura del diagrama de caja del navegador con las medidas reales de las cuatro áreas sobre un elemento del proyecto

Figura 2.7. El inspector del modelo de caja

Dos comprobaciones más, que se usan poco. **La verificación de contraste** está en el selector de color e indica si la combinación de texto y fondo alcanza el mínimo de las WCAG 2.2. Y **la emulación de preferencias** fuerza `prefers-reduced-motion` o el esquema oscuro **sin tocar la configuración del sistema**.

2.13 — Seguridad y evolución

CSS no tiene la superficie de ataque de JavaScript, pero **tampoco es inocuo**.

CSS puede exfiltrar información

Un selector de atributo combinado con una imagen de fondo permite **deducir el contenido de un campo carácter por carácter**: una regla por carácter posible, cada una con una imagen de fondo en

un servidor externo, y cuando el valor coincide el navegador pide la imagen — y esa petición delata el carácter. **No requiere JavaScript.** Por eso el Content-Security-Policy de la **sección 16.5 del TPI** —la defensa contra el código inyectado del Capítulo 1— **también restringe los orígenes de las hojas de estilo, no sólo los de los scripts.**

CSS puede destruir la accesibilidad

Es **el riesgo real y el más frecuente**, y no requiere ningún atacante. Tres formas de lograrlo sin querer:

Lo que hacés	Qué rompe	Qué hacer
Eliminar el foco con <code>outline: none</code>	Quien navega con teclado pierde toda referencia de dónde está	Se reemplaza con <code>:focus-visible</code> ; nunca se elimina
Ocultar con <code>display: none</code> cuando se quería ocultar sólo visualmente	Ese contenido desaparece también para los lectores de pantalla	El patrón <i>visually hidden</i>
Reordenar visualmente con <code>order</code> o <code>Grid</code>	El orden de tabulación sigue el DOM, no el visual	Reordenar en el marcado, no en la presentación

OJO ACÁ: la interfaz que se ve bien y se recorre mal

El tercer caso es el más difícil de detectar porque **en la pantalla no se ve nada raro**. Si moviste el tercer elemento al primero con `order: -1`, para el teclado sigue siendo el tercero — **y vos no te enterás nunca, porque usás el mouse.**

La comprobación dura un minuto: **recorré tu interfaz con Tab y mirá adónde va el foco**. Si el recorrido no tiene sentido leído en voz alta, está mal.

Cuatro incorporaciones recientes, y qué problema cierra cada una

La incorporación	Qué permite, y qué problema cierra
Propiedades personalizadas (variables de CSS)	Valores nombrados que se heredan y se leen y modifican desde JavaScript. Las de los preprocesadores no podían: se resolvían antes del navegador
Capas en cascada (<code>@layer</code>)	Un nivel más en el algoritmo de la sección 2.5, entre el origen y la especificidad : que unas reglas siempre le ganen a otras sin especificidad ni !important — la deuda de la sección 2.5.4
Consultas de contenedor (<code>@container</code>)	Que un componente responda al ancho de su contenedor y no al de la ventana: distinto en la barra lateral y en el cuerpo
La pseudoclase <code>:has()</code>	Seleccionar un elemento según lo que contiene : «el selector del padre» que se pidió veinte años

2.14 — Verificación: el checklist honesto

Nueve comprobaciones. **No son ejercicios: son el criterio para saber si el capítulo se entendió.**

- Abrir el inspector sobre un elemento cualquiera y **enumerar las reglas que le aplican en orden de prioridad**, identificando cuál ganó y por qué. (2.5)
- Encontrar una declaración tachada y **explicar qué criterio de la cascada la dejó afuera**. (2.5)

- Calcular a mano la especificidad de tres selectores del proyecto y **verificarla en el inspector**. (2.5.2)
- Medir un elemento con el diagrama del modelo de caja y **verificar la suma** de contenido, relleno y borde según el valor de `box-sizing`. (2.6.2)
- Producir deliberadamente un colapso de márgenes entre padre e hijo y **neutralizarlo de dos maneras distintas**. (2.6.3)
- Construir una fila con Flexbox y **comprobar qué propiedades cambian de efecto** al pasar `flex-direction` de `row` a `column`. (2.8)
- Construir una grilla con `auto-fill` y `minmax` y verificar **cuántas columnas genera en tres anchos distintos**. (2.9)
- Cambiar el tamaño de fuente base del navegador a 24 píxeles y **verificar que la interfaz propia sigue siendo utilizable**. (2.10)
- Recorrer la interfaz propia con la tecla **Tab** y confirmar que el indicador de foco es visible en todos los elementos interactivos. (2.13)

2.15 — Los once errores frecuentes

Todos tienen algo en común: **en el momento, no parecen errores**. Por eso son frecuentes, y en CSS son especialmente traicioneros porque ninguno produce mensaje.

El error	Por qué duele	Sección
Buscar errores de CSS en la consola	No hay ninguno: el lenguaje descarta en silencio, y el lugar correcto es el panel de estilos	2.1 y 2.4
Escalar la guerra de <code>!important</code>	Se pone uno para ganarle a una regla que no se entiende, y el siguiente necesita otro	2.5.4
Creer que la especificidad se acumula	Diez clases no le ganan a un identificador: es lexicográfica, no numérica	2.5.2
Olvidar <code>box-sizing</code> : <code>border-box</code>	Desbordes que se atribuyen a otras causas, sobre todo en anchos porcentuales	2.6.2
Sumar márgenes verticales que colapsan	La separación no coincide con la esperada y se compensa con más margen, lo que lo empeora	2.6.3
<code>width</code> o <code>margin-top</code> en un elemento en línea	Se ignoran sin aviso: la declaración aparece aplicada y no hace nada	2.7.1
Posicionar en absoluto sin ancestro posicionado	El elemento se ubica respecto del documento y aparece en cualquier parte	2.7.3
Confundir <code>justify-content</code> con <code>align-items</code>	Se refieren a ejes, no a direcciones fijas , y su efecto se invierte con <code>column</code>	2.8
Clases de Tailwind por concatenación	El escaneo no la encuentra y su CSS no se genera: anda en desarrollo y falla en producción	2.11.2

El error	Por qué duele	Sección
Fijar la tipografía en píxeles	Anula la preferencia del usuario, que es la segunda decisión de diseño	2.10
Eliminar el contorno de foco	Deja inutilizable la navegación por teclado , y no se nota si usás mouse	2.13

2.16 — Las actividades, y qué busca cada una

Siete actividades, y debajo de cada una lo que quiere que descubras.

1. Auditoría de cascada

Elegir tres elementos de un sitio real y documentar, para la propiedad `color` de cada uno, **todas las declaraciones que compiten**, cuál ganó y en qué criterio de la sección 2.5 se decidió.

Qué busca: *ver que la especificidad muchas veces ni se mira.*

2. Especificidad a mano

Calcular el vector de especificidad de ocho selectores dados, ordenarlos y verificar el orden **en un documento donde todos apunten al mismo elemento con colores distintos**.

Qué busca: *verificación empírica y no de fe: vas a fallar en alguno, y ese enseña.*

3. El modelo de caja medido

Construir una tarjeta de producto con ancho, relleno y borde declarados, medirla con `box-sizing: content-box`, cambiar a `border-box` y volver a medir. **Explicar la diferencia con la suma.**

Qué busca: *que el 344 de la sección 2.6.2 sea un número que mediste vos.*

4. Catálogo adaptable

Maquetar una grilla de tarjetas que pase de una a cuatro columnas según el ancho disponible, **sin consultas de medios**, con `auto-fill` y `minmax`. Documentar en qué anchos cambia el número de columnas.

Qué busca: *la diferencia entre dar órdenes y declarar condiciones: lo segundo cubre anchos que nunca probaste.*

5. Traducción de Tailwind

Tomar un componente escrito con doce utilidades de Tailwind y **escribir el CSS equivalente con una sola clase**. Comparar los dos en líneas y en facilidad para modificar un valor.

Qué busca: *que Tailwind es CSS y nada más, y que el intercambio de la sección 2.11.3 es real en las dos direcciones.*

6. Exploración: la preferencia del usuario

Configurar el navegador con 24 píxeles de fuente y auditar tres sitios de uso cotidiano, documentando cuál escala, cuál no y cuál se rompe. **Relacionarlo con la segunda decisión de diseño de la sección 2.2** y explicar qué unidad usó cada uno. *(Requiere revertir la configuración al terminar.)*

Qué busca: *que la decisión de 1996 sea algo que viste romperse en tres sitios reales.*

7. Exploración: el costo del reordenamiento visual

Construir una fila con Flexbox de cinco elementos interactivos, reordenarlos con `order`, recorrerla con Tab y documentar el recorrido. **Relacionarlo con la afirmación de la sección 2.13 sobre el orden de tabulación** y proponer una solución que no dependa de `order`. (Requiere navegación por teclado.)

Qué busca: una interfaz que se ve impecable y se recorre como un juego de la oca: lo que un usuario de teclado encuentra todos los días.

2.17 — Síntesis: las once frases

1. CSS existe porque **mezclar presentación y estructura destruye la estructura**. Las etiquetas de presentación de los noventa produjeron documentos ilegibles para toda herramienta que no fuera un navegador gráfico: la pérdida que el Capítulo 1 describió al hablar del árbol de accesibilidad.
2. La decisión de diseño más importante del lenguaje es que **el control está repartido entre navegador, usuario y autor**, y que en caso de conflicto marcado como importante **gana el usuario**. Todo lo que impide esa adaptación —empezando por la tipografía en píxeles— trabaja en contra del diseño del lenguaje.
3. **CSS nunca falla ruidosamente**: lo inválido se descarta en silencio, sin excepciones ni errores de compilación. Por eso el trabajo con CSS es principalmente diagnóstico, y el panel de estilos es la herramienta central.
4. La cascada resuelve conflictos en un orden fijo: **origen e importancia, luego capas, luego especificidad, y por último orden de aparición**. La especificidad no es lo primero, aunque casi todos lo crean.
5. La especificidad **se compara lexicográficamente, no se suma**. Un identificador le gana a cualquier cantidad de clases, y por eso `!important` no es una solución sino una deuda que el siguiente va a tener que pagar.
6. `width` **no define lo que el elemento ocupa** salvo que se declare `box-sizing: border-box`. El modelo que hoy se considera correcto fue durante años el error de Internet Explorer.
7. Los márgenes verticales **colapsan**: se fusionan tomando el mayor. Es una propiedad del flujo normal, y por eso desaparece dentro de un contenedor flex o grid, que establecen contextos de formato distintos.
8. **display no cambia una propiedad: cambia el conjunto de reglas** bajo el cual el elemento y sus hijos se disponen. Eso explica por qué cambia tantas cosas.
9. Flexbox y Grid no compiten: **uno parte del contenido en un eje, el otro del contenedor en dos**. Se usan juntos, Grid para la página y Flexbox para el interior de los componentes.
10. Tailwind no resuelve un problema de estética sino de **crecimiento sin límite de las hojas de estilo**: resigna legibilidad del marcado a cambio de que agregar componentes deje de agregar CSS. Y no exime de saber CSS, exime de nombrar.
11. **El orden de tabulación sigue el DOM, no lo visual**. Todo reordenamiento hecho con CSS que altere el orden lógico produce una interfaz que se ve bien y se recorre mal.

2.18 — Qué leer, y en qué orden

El original las lista en dos párrafos densos. Acá van ordenadas por prioridad real.

Si lees una sola cosa

Andrew, *The New CSS Layout* (A Book Apart, 2017). Explica Flexbox y Grid **desde el problema que resuelven en lugar de desde su sintaxis**, que es el enfoque de este capítulo. Te deja el modelo mental; la sintaxis después se busca.

Si lees tres

- **Meyer y Weyl**, *CSS: The Definitive Guide* (5.ª edición, O'Reilly, 2023): la referencia completa del lenguaje, **la más adecuada para consultar un comportamiento puntual con precisión normativa**. Para tener al lado, no para leer de corrido.
- **Pickering**, *Inclusive Components* (2018): patrones de componentes que funcionan con teclado y lector de pantalla, **la mejor respuesta práctica a los riesgos de la sección 2.13**.
- **La documentación de Tailwind**, en tailwindcss.com, con el artículo de su autor sobre el razonamiento detrás del enfoque *utility-first* de la sección 2.11.

Las fuentes normativas (para consultar, no para leer de corrido)

- **Los módulos del W3C**. La especificación de CSS no es un documento único sino un conjunto de módulos independientes, accesibles desde www.w3.org/Style/CSS/. Los pertinentes acá: **CSS Cascading and Inheritance Level 5**, que define el algoritmo de la sección 2.5 y las capas en cascada; **CSS Display Level 3**, que formaliza el flujo normal y los contextos de formato de la sección 2.7; **CSS Box Model Level 3** y **CSS Box Sizing Level 3**, para el modelo de caja y `box-sizing`; **CSS Flexible Box Layout Level 1** y **CSS Grid Layout Level 2**, para las secciones 2.8 y 2.9; **CSS Values and Units Level 4**, para las unidades de la sección 2.10; y **CSS Containment Level 3**, que introduce las consultas de contenedor de la sección 2.13.
- **La propuesta original**. El texto de Håkon Wium Lie de octubre de 1994 sigue disponible en www.w3.org/People/howcome/p/cascade.html, y su lectura muestra con claridad **qué problema se intentaba resolver y qué se descartó**. Es corto.
- **Accesibilidad**. Las pautas citadas a lo largo del capítulo son las **WCAG 2.2**, en particular sus criterios de **contraste mínimo** y de **visibilidad del foco** — los dos que la sección 2.13 puede romper sin que nadie se entere.

Cierre: las siete cosas que hay que recordar

Si dentro de un mes te acordás de siete frases, que sean estas.

LAS SIETE

1. **CSS no avisa nada**. No hay consola, no hay excepción, no hay compilador. Cuando algo no se ve bien, la pregunta no es «¿cómo lo arreglo?» sino «¿qué regla ganó?».
2. **La última palabra no es tuya, es del que lee**. Esa es la segunda decisión de diseño del lenguaje, y `font-size` en píxeles la rompe.
3. **La especificidad no se suma, se compara por columnas**. Un identificador le gana a cualquier cantidad de clases. Es 0,34 contra 1,00.
4. **width no es lo que ocupa**, salvo con `border-box`. El bug de Internet Explorer terminó siendo el estándar de hecho.
5. **Los márgenes verticales no se suman: se acuerdan**. El espacio entre dos bloques es de los dos, no de cada uno.
6. **display cambia el reglamento, no una propiedad**. Por eso `flex` y `grid` arrastran consecuencias en cosas que ni tocaste.
7. **El Tab sigue al DOM, no a lo que ves**. Una interfaz puede verse impecable y recorrerse como un juego de la oca.

Y una octava, que no está escrita pero está en todas sus páginas: **cuando el CSS no hace lo que esperabas, el CSS funcionó**. Lo que falló fue tu modelo de cuáles eran las reglas, y el inspector está a un F12 de distancia para mostrarte cuál era. Eso separa maquetar de adivinar, y es lo que el Capítulo 3 va a necesitar cuando empecemos a mover estas cajas por programa.